Process & Decision Documentation

Project/Assignment Decisions

Side Quests and A4 (Individual Work)

This assignment focused on creating debug screens and making users be able to toggle features on the screen. Was able to figure out how to edit the game to be able to have different difficulty levels, being able to provide more accessibility, giving choice to players to cater to different skill levels.

Role-Based Process Evidence

Entry Header

Name: Victory Okusanya

Role(s): Creator

Primary responsibility for this work:

Goal of Work Session

Briefly describe what you were trying to accomplish during this phase of the assignment.

Examples:

- Creating debug screen
- Creating toggleable features

Tools, Resources, or Inputs Used

- GenAI tools: Claude, Copilot
- Prior drafts or code
- Playtesting feedback

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: March 26, 2026

Tool Disclosure: CoPilot and Claude.ai

Purpose of Use: Support with coding efforts

Summary of Interaction: The tools contributed to the creation of the debug screen as well as the different modes that were toggleable within the debug screen.

Human Decision Point(s): Decided what the different modes would be, describing the different features for each togglable challenge mode.

Integrity & Verification Note: I looked over the good and playtested the game, making sure that it worked how I intended.

Scope of GenAI Use: The GenAi didn't contribute to the idea of the what would be toggled in the screen in terms of the challenge mode.

Limitations or Misfires: I ran out of co-polit usage for the month so had to move to claude.ai due to limited understanding and knowledge of coding.

Summary of Process (Human + Tool)

- Asked for the creation of a debug screen.
- Asked for different modes possible within the toggle screen.

Decision Points & Trade-offs

Describe one or two key decisions you made:

- Be more specific with what different changes I wanted to ideal outcome.
- Changed to using Claude ai even though it would be able to reference of the file that co-pilot has.

Verification & Judgement

Explain how you evaluated whether your decision or change was appropriate:

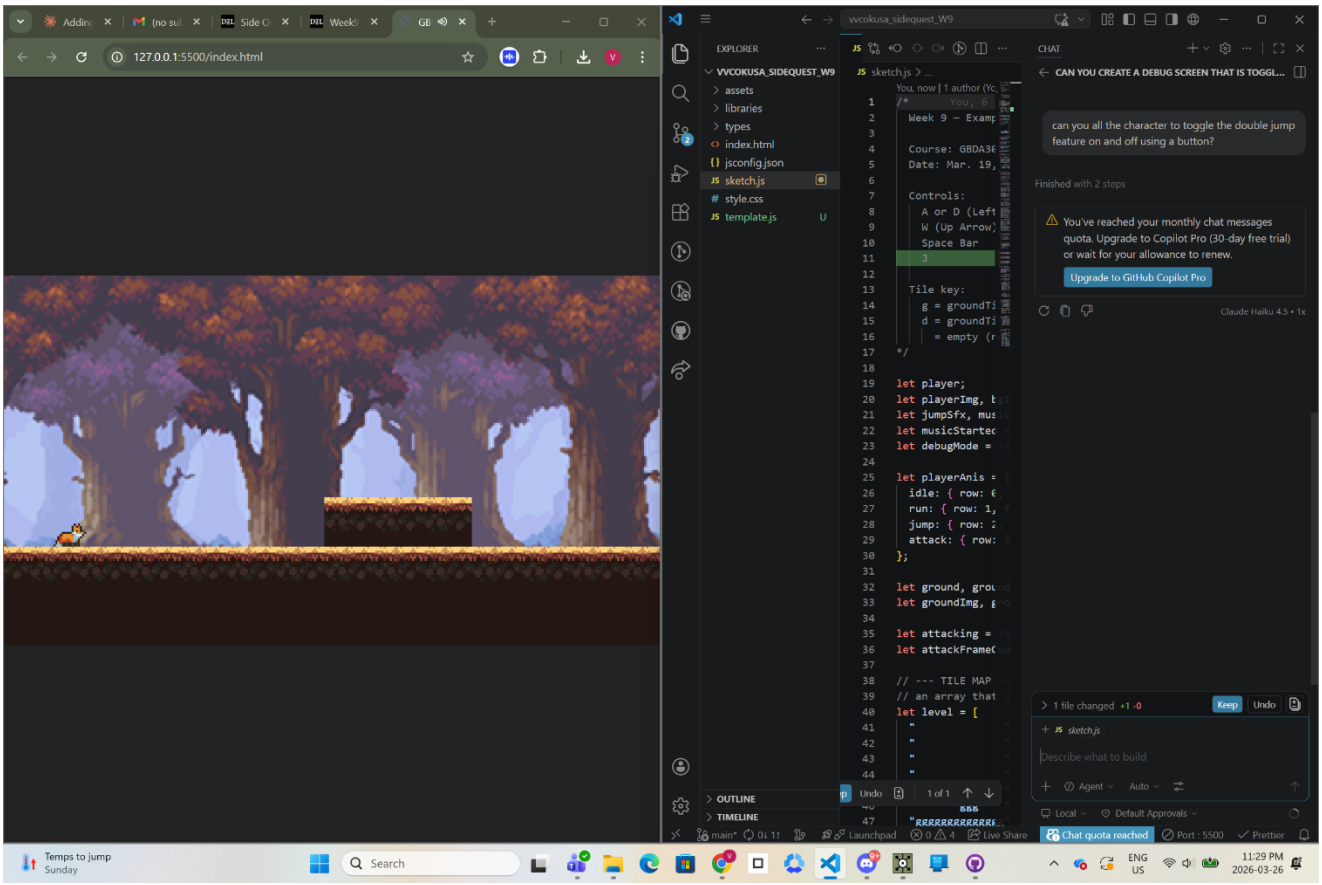
- Playtesting

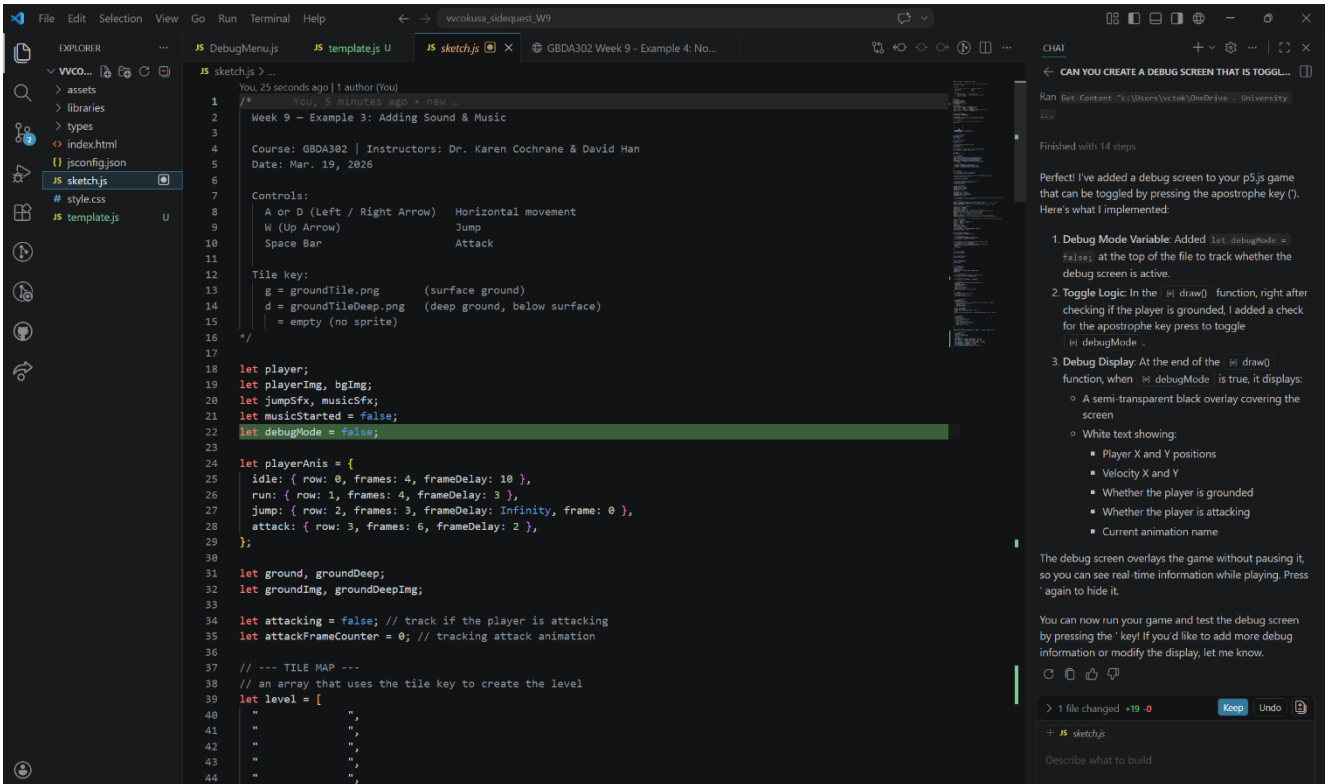
Limitations, Dead Ends, or Open Questions

Note anything that:

- Did not work as expected had to pivot.
- Was left unresolved
- Had to change Ai models because Co-polit ran out of usage.

Appendix





ME: can you help make it so that debug toggle screen allows the player to be able to toggle the challenge mode of the game, making it so that the current game mode is easy, medium game mode made the playable character harder to move and added gaps in the ground, and the hard mode made the playable character harder to move, added gaps in the ground, and added a visibility radius where the character can now only see a smaller circle area of the game surrounding the player character?

Claude:

Here's the plan before I write the code:

- **Easy** (default): normal speed (1.5), full ground, full visibility
- **Medium**: slower speed (0.8), gaps rebuilt into the ground row, full visibility
- **Hard**: slowest speed (0.4), same gaps, plus a dark fog-of-war overlay with a circular "torch light" cutout around the player
- The difficulty cycles with **M** key while in debug mode, and the current mode is shown in the debug overlay
- Changing difficulty rebuilds the ground tiles (removes existing ones, lays new Tiles)

The fog effect is drawn using a p5 graphics buffer as a mask — a black rectangle with a radial gradient hole punched out at the player's screen position.

Created a file, read a file

Created a file, read a file

Here's what was added and how it all works:

Difficulty system (difficulty variable, 0/1/2)

- Cycles with **M** — but only while the debug screen is open (' to toggle), so players can't accidentally change it mid-game.
- MOVE_SPEEDS = [1.5, 0.8, 0.4] maps directly to Easy/Medium/Hard, applied when setting player.vel.x.

Medium & Hard — ground gaps (levelHard)

A second tile map string is defined with spaces punched into both the surface (g) and deep (d) rows at two spots. When you switch to/from Easy, rebuildLevel() removes all existing tile

sprites and lays a fresh Tiles object from the correct map. The player is also repositioned to the safe left side so they don't spawn over a gap.

Hard — fog of war (drawFog())

A p5.Graphics buffer (fogBuffer) is redrawn each frame:

1. Fill it with a dark semi-transparent black.
2. Switch to blendMode(LIGHTEST) and paint concentric white circles from the player's position outward — the white "erases" the black from the centre out, creating a smooth radial fade.
3. The buffer is stamped onto the canvas after all world sprites are drawn but before the debug overlay, so the debug UI is always readable.

Debug overlay additions

The existing stats remain, and below them you'll see the three difficulty labels with the active one highlighted in colour (green/yellow/red), plus a one-line description of what each mode does.

Here's the plan before I write the code:

- **Easy** (default): normal speed (1.5), full ground, full visibility
- **Medium**: slower speed (0.8), gaps rebuilt into the ground row, full visibility
- **Hard**: slowest speed (0.4), same gaps, plus a dark fog-of-war overlay with a circular "torch light" cutout around the player
- The difficulty cycles with **M** key while in debug mode, and the current mode is shown in the debug overlay
- Changing difficulty rebuilds the ground tiles (removes existing ones, lays new Tiles)

The fog effect is drawn using a p5 graphics buffer as a mask — a black rectangle with a radial gradient hole punched out at the player's screen position.

Created a file, read a file

Created a file, read a file

Here's what was added and how it all works:

Difficulty system (difficulty variable, 0/1/2)

- Cycles with **M** — but only while the debug screen is open (' to toggle), so players can't accidentally change it mid-game.
- MOVE_SPEEDS = [1.5, 0.8, 0.4] maps directly to Easy/Medium/Hard, applied when setting player.vel.x.

Medium & Hard — ground gaps (levelHard)

A second tile map string is defined with spaces punched into both the surface (g) and deep (d) rows at two spots. When you switch to/from Easy, rebuildLevel() removes all existing tile sprites and lays a fresh Tiles object from the correct map. The player is also repositioned to the safe left side so they don't spawn over a gap.

Hard — fog of war (drawFog())

A p5.Graphics buffer (fogBuffer) is redrawn each frame:

1. Fill it with a dark semi-transparent black.
2. Switch to blendMode(LIGHTEST) and paint concentric white circles from the player's position outward — the white "erases" the black from the centre out, creating a smooth radial fade.
3. The buffer is stamped onto the canvas after all world sprites are drawn but before the debug overlay, so the debug UI is always readable.

Debug overlay additions

The existing stats remain, and below them you'll see the three difficulty labels with the active one highlighted in colour (green/yellow/red), plus a one-line description of what each mode does.

```
/*
 Week 9 — Example 3: Adding Sound & Music


 Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han

 Date: Mar. 19, 2026


 Controls:
```

```
  A or D (Left / Right Arrow)   Horizontal movement
  W (Up Arrow)              Jump
  Space Bar                 Attack
  ' (apostrophe)            Toggle Debug Screen
  M (while debug open)        Cycle Difficulty  (Easy → Medium → Hard → Easy)

 Difficulty modes:
  Easy   — normal movement speed, full ground, full visibility
  Medium — slower movement speed, gaps in the ground
  Hard   — slowest movement speed, gaps in the ground,
           fog-of-war with a small visibility radius around the player

 Tile key:
  g = groundTile.png      (surface ground)
  d = groundTileDeep.png   (deep ground, below surface)
    = empty (no sprite)
*/

let player;
let playerImg, bgImg;
let jumpSfx, musicSfx;
let musicStarted = false;
let debugMode = false;

let playerAnis = {
 idle:   { row: 0, frames: 4, frameDelay: 10 },
```

```
  run:   { row: 1, frames: 4, frameDelay: 3  },
  jump:  { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2  },
};

let ground, groundDeep;
let groundImg, groundDeepImg;

let attacking       = false;
let attackFrameCounter = 0;

// --- DIFFICULTY ---
// 0 = Easy  |  1 = Medium  |  2 = Hard
let difficulty      = 0;
const DIFF_NAMES    = ["Easy", "Medium", "Hard"];

// Movement speeds per difficulty
const MOVE_SPEEDS   = [1.5, 0.8, 0.4];

// Fog-of-war graphics buffer (used in Hard mode)
let fogBuffer;
// Visibility radius in game-pixels for Hard mode
const VIS_RADIUS    = 55;

// --- TILE MAPS ---
// Easy / default — solid ground rows
```

```
const levelEasy = [
  "            ",
  "            ",
  "            ",
  "            ",
  "            ",
  "      ggg   ",
  "gggggggggggggg",
  "ddddddddddddd",
];

// Medium & Hard — gaps punched into the surface and deep rows
const levelHard = [
  "            ",
  "            ",
  "            ",
  "            ",
  "            ",
  "      ggg   ",
  "ggg  ggg  gggg",   // gaps at columns 3-4 and 9-10
  "ddd  ddd  dddd",
];

// --- LEVEL CONSTANTS ---
const VIEWW = 320, VIEWH = 180;
const TILE_W = 24,  TILE_H = 24;
```

```
const FRAME_W = 32, FRAME_H = 32;
const MAP_START_Y = VIEWH - TILE_H * 4;
const GRAVITY = 10;

// --- SENSOR (declared here so rebuildLevel can reference it) ---
let sensor;

// ────────────────────────────────────────────────────────
function preload() {
  playerImg    = loadImage("assets/foxSpriteSheet.png");
  bgImg        = loadImage("assets/combinedBackground.png");
  groundImg    = loadImage("assets/groundTile.png");
  groundDeepImg = loadImage("assets/groundTileDeep.png");

  if (typeof loadSound === "function") {
    jumpSfx  = loadSound("assets/sfx/jump.wav");
    musicSfx = loadSound("assets/sfx/music.wav");
  }
}

// ────────────────────────────────────────────────────────
function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;
```

```
if (musicSfx) musicSfx.setLoop(true);
startMusicIfNeeded();

// --- TILE GROUPS ---
ground = new Group();
ground.physics = "static";
ground.img     = groundImg;
ground.tile    = "g";

groundDeep = new Group();
groundDeep.physics = "static";
groundDeep.img     = groundDeepImg;
groundDeep.tile    = "d";

new Tiles(levelEasy, 0, 0, TILE_W, TILE_H);

// --- PLAYER ---
player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H);
player.spriteSheet   = playerImg;
player.rotationLock  = true;
player.anis.w       = FRAME_W;
player.anis.h       = FRAME_H;
player.anis.offset.y = -4;
player.addAnis(playerAnis);
player.ani       = "idle";
player.w         = 18;
```

```
  player.h        = 20;
  player.friction   = 0;
  player.bounciness = 0;

  // --- GROUND SENSOR ---
  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;

  // --- FOG BUFFER ---
  // Created at native resolution; we'll redraw it each frame in Hard mode.
  fogBuffer = createGraphics(VIEWW, VIEWH);
}

// ————————————————————————————————————————————
// Rebuild ground tiles when difficulty changes.
// Removes every sprite in both tile groups and lays a fresh Tiles object.
function rebuildLevel() {
  // Remove all existing ground sprites
```

```
  for (let i = ground.length - 1; i >= 0; i--)  ground[i].remove();
  for (let i = groundDeep.length - 1; i >= 0; i--) groundDeep[i].remove();

  const map = (difficulty === 0) ? levelEasy : levelHard;
  new Tiles(map, 0, 0, TILE_W, TILE_H);

  // Reset player to a safe position above the left side of the map
  player.pos.x = FRAME_W;
  player.pos.y = MAP_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;
}

// ————————————————————————————————————————————
// Draw the fog-of-war overlay for Hard mode.
// We use a p5.Graphics buffer: fill it black, then "erase" a circle
// around the player using a radial gradient drawn with blendMode(LIGHTEST).
function drawFog() {
  // Convert player world position to screen coordinates.
  // Since the camera is centred on the canvas (no scrolling in this demo)
  // the world coords == screen coords directly.
  let px = player.pos.x;
  let py = player.pos.y;

  fogBuffer.clear();
  fogBuffer.background(0, 0, 0, 220); // dark fog, slight transparency
```

```
  // Punch a radial "light" hole: draw concentric circles from transparent
  // in the centre fading to the fog colour at the edge, using LIGHTEST blend.
  fogBuffer.push();
  fogBuffer.blendMode(LIGHTEST); // lighten = effectively erases the black
  for (let r = VIS_RADIUS; r > 0; r -= 1) {
    let t = r / VIS_RADIUS;                // 1 at edge, 0 at centre
    let alpha = lerp(255, 0, t);           // opaque at centre, transparent at edge
    fogBuffer.noStroke();
    fogBuffer.fill(255, 255, 255, alpha);
    fogBuffer.ellipse(px, py, r * 2, r * 2);
  }
  fogBuffer.pop();

  // Draw the buffer on top of the world (camera already off from caller)
  image(fogBuffer, 0, 0);
}

// ————————————————————————————————————————————
function startMusicIfNeeded() {
  if (musicStarted || !musicSfx) return;
  const startLoop = () => {
    if (!musicSfx.isPlaying()) musicSfx.play();
    musicStarted = musicSfx.isPlaying();
  };
  const maybePromise = userStartAudio();
```

```
  if (maybePromise && typeof maybePromise.then === "function") {
    maybePromise.then(startLoop).catch(() => {});
  } else {
    startLoop();
  }
}

function keyPressed()   { startMusicIfNeeded(); }
function mousePressed() { startMusicIfNeeded(); }
function touchStarted() { startMusicIfNeeded(); return false; }

// ————————————————————————————————————————————
function draw() {
  // --- BACKGROUND ---
  camera.off();
  imageMode(CORNER);
  image(bgImg, 0, 0, bgImg.width, bgImg.height);
  camera.on();

  // --- PLAYER CONTROLS ---
  let grounded = sensor.overlapping(ground);

  // -- DEBUG TOGGLE --
  if (kb.presses("'")) debugMode = !debugMode;

  // -- DIFFICULTY CYCLE (only while debug is open) --
```

```
if (debugMode && kb.presses("m")) {
  let prev = difficulty;
  difficulty = (difficulty + 1) % 3;
  // Rebuild the level if the ground layout needs to change
  if (prev === 0 || difficulty === 0) rebuildLevel();
}

// -- ATTACK INPUT --
if (grounded && !attacking && kb.presses("space")) {
  attacking = true;
  attackFrameCounter = 0;
  player.vel.x = 0;
  player.ani.frame = 0;
  player.ani = "attack";
  player.ani.play();
}

// -- JUMP --
if (grounded && kb.presses("up")) {
  player.vel.y = -4;
  if (jumpSfx) jumpSfx.play();
}

// --- STATE MACHINE ---
if (attacking) {
  attackFrameCounter++;
```

```
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT (speed varies by difficulty) ---
let spd = MOVE_SPEEDS[difficulty];
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -spd;
    player.mirror.x = true;
  } else if (kb.pressing("right")) {
    player.vel.x = spd;
    player.mirror.x = false;
  }
}

// --- KEEP IN VIEW ---
player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2);
```

```
// --- HARD MODE FOG OF WAR ---
if (difficulty === 2) {
  camera.off();
  drawFog();
  camera.on();
}

// --- DEBUG SCREEN ---
if (debugMode) {
  camera.off();
  fill(0, 0, 0, 160);
  noStroke();
  rect(0, 0, VIEWW, VIEWH);

  fill(255);
  textSize(8);

  // Stats
  text(`Player X:   ${player.x.toFixed(2)}`,      10, 16);
  text(`Player Y:   ${player.y.toFixed(2)}`,      10, 27);
  text(`Velocity X: ${player.vel.x.toFixed(2)}`,  10, 38);
  text(`Velocity Y: ${player.vel.y.toFixed(2)}`,  10, 49);
  text(`Grounded:   ${grounded}`,                 10, 60);
  text(`Attacking:  ${attacking}`,                10, 71);
  text(`Animation:  ${player.ani.name}`,          10, 82);
```

```
// Difficulty display
let diffColours = ["#44ff88", "#ffdd44", "#ff5555"];
let diffCol     = diffColours[difficulty];
textSize(8);
fill(180);
text("DIFFICULTY:", 10, 100);

// Highlight the active difficulty, dim the others
let labels = ["[Easy]", "[Medium]", "[Hard]"];
let xPos   = [10, 55, 115];
for (let i = 0; i < 3; i++) {
  if (i === difficulty) {
    fill(diffCol);
    textSize(9);
  } else {
    fill(100);
    textSize(8);
  }
  text(labels[i], xPos[i], 112);
}

// Instruction hint
fill(160);
textSize(7);
text("Press M to cycle difficulty", 10, 125);
```

```
    // Per-mode description
    let descs = [
      "Normal speed | Solid ground | Full vision",
      "Slow speed | Gaps in ground | Full vision",
      "Slowest speed | Gaps in ground | Limited vision",
    ];
    fill(200);
    textSize(7);
    text(descs[difficulty], 10, 136);

    camera.on();
  }
}
```